

GESTION PLANNING

Document Technique & Fonctionnel

Spécifications techniques et fonctionnelles complètes

Version : 1.0

Date : Avril 2026

Auteur : Al hussein Khouma

Backend : Node.js 22 + Express 5 + Prisma + PostgreSQL

Frontend : React 19 + Vite + Tailwind CSS 4 + Zustand + React Router 7

Temps réel : Socket.IO | Email : Nodemailer SMTP

1. Présentation Générale du Projet.....	5
1.1 Contexte et objectifs	5
1.2 Périmètre fonctionnel	5
1.3 Contraintes et exigences non-fonctionnelles	5
2. Architecture Technique	6
2.1 Vue d'ensemble	6
2.2 Structure du projet	6
Backend (D:/Gestion planning/backend/).....	6
Frontend (D:/Gestion planning/frontend/src/).....	6
2.3 Flux de communication	7
3. Modèle de Données (Base de Données)	8
3.1 Modèles Utilisateurs & Authentification	8
User.....	8
RefreshToken.....	8
PasswordHistory	8
DeviceToken (push mobile)	9
3.2 Modèles Planning	9
Planning	9
PlanningEvent.....	9
3.3 Modèles Réunions	10
Meeting	10
Invitation.....	10
3.4 Modèles Missions	11
Mission	11
MissionAssignment.....	11
3.5 Modèles Salles	11
Room	11
RoomBooking	11
3.6 Modèles Messagerie.....	12
DirectMessage	12
DirectionMessage	12
MeetingMessage	12
3.7 Modèles Organisationnels	13
Direction	13
Project.....	13
3.8 Modèles Système	13
Notification	13
AuditLog.....	14

AppSetting	14
Backup	14
4. API REST — Endpoints par Module	15
4.1 Authentification — /api/auth	15
4.2 Double Authentification — /api/2fa	15
4.3 Profil — /api/profile	15
4.4 Plannings — /api/plannings	16
4.5 Réunions — /api/meetings	16
4.6 Missions — /api/missions	17
4.7 Salles — /api/rooms	17
4.8 Projets — /api/projects	17
4.9 Messagerie Directe — /api/direct-messages	18
4.10 Notifications — /api/notifications	18
4.11 Administration — /api/users & /api/admin	18
4.12 Super Admin — /api/super-admin	19
5. Fonctionnement Temps Réel — Socket.IO	20
5.1 Architecture Socket.IO	20
5.2 Événements émis par le serveur	20
5.3 Événements émis par le client	20
6. Authentification & Sécurité	21
6.1 Flux d'authentification JWT	21
6.2 Gestion des tokens	21
6.3 Middleware d'authentification	21
6.4 Contrôle d'accès basé sur les rôles (RBAC)	21
6.5 Protection brute-force	21
6.6 Headers de sécurité (Helmet.js)	22
7. Système de Notifications	23
7.1 Canaux de notification	23
7.2 Préférences utilisateur	23
7.3 Jobs automatiques (node-cron)	23
8. Gestion des Fichiers (Upload)	24
9. Configuration — Variables d'Environnement	25
9.1 Variables Backend (.env)	25
9.2 Variables Frontend (.env)	25
10. Logging & Observabilité	27
10.1 Loggers Winston	27
10.2 Middleware de logging des requêtes	27
10.3 Audit Trail (AuditLog)	27

11. Workflows Métier Détaillés	28
11.1 Workflow de Validation des Plannings.....	28
11.2 Workflow d'Organisation d'une Réunion	28
11.3 Workflow d'Activation d'un Compte	28
12. Déploiement & Infrastructure	29
12.1 Prérequis.....	29
12.2 Installation et démarrage	29
12.3 Ports et URLs.....	29
12.4 Migrations de base de données	29
13. Glossaire	30

1. Présentation Générale du Projet

1.1 Contexte et objectifs

Gestion Planning est une plateforme web d'entreprise permettant à une organisation de gérer l'ensemble de son cycle d'activité : planification hebdomadaire des équipes, organisation de réunions, suivi des missions terrain, gestion de projets et communication interne. L'application vise à remplacer les outils bureautiques dispersés (Excel, email) par une solution collaborative centralisée, sécurisée et traçable.

1.2 Périmètre fonctionnel

- Planification hebdomadaire avec workflow de validation multi-niveaux (RESPONSABLE → CONSOLIDATEUR → DG)
- Organisation de réunions avec invitations, messagerie intégrée et réservation de salles
- Gestion de missions professionnelles (déplacements, formations, interventions)
- Suivi de projets avec documents associés
- Messagerie interne : directe (1-à-1), par direction (groupe), par réunion
- Notifications en temps réel (in-app via Socket.IO, push via Firebase, email via SMTP)
- Administration complète : utilisateurs, rôles, paramètres, audit, sauvegardes

1.3 Contraintes et exigences non-fonctionnelles

Contrainte	Description
Sécurité	Authentification JWT, 2FA TOTP, protection brute-force, audit trail complet, headers Helmet.js
Performance	API REST < 200ms (P95), Socket.IO temps réel, rate limiting 200 req/min
Disponibilité	Architecture stateless (backend) scalable horizontalement, sauvegardes hebdomadaires automatiques
Traçabilité	Audit log de toutes les actions : qui, quoi, quand, depuis quelle IP
Conformité CDC §3	Historique mots de passe (3 derniers), suppression logique, rate limiting login/2FA
PWA	Application installable mobile, notifications push FCM, bannière hors-ligne

2. Architecture Technique

2.1 Vue d'ensemble

L'application suit une architecture client-serveur classique en deux tiers, enrichie de communications temps réel et de services tiers.

Couche	Technologie	Rôle
Frontend	React 19 + Vite	Interface utilisateur SPA, state management, routing côté client
API REST	Express.js 5	Endpoints CRUD, authentification, middleware chain
Temps réel	Socket.IO 4	Présence, messagerie, notifications push in-app
Base de données	PostgreSQL (Prisma ORM)	Persistance, relations, migrations
Cache / Sessions	JWT (localStorage)	Tokens access (15min) + refresh (7j) sans session serveur
Emails	Nodemailer + SMTP	Notifications, invitations, reset password, rapports
Push mobile	Firebase FCM	Notifications push Android/iOS/Web (PWA)
Fichiers	express.static + multer	Upload et service des fichiers statiques
Jobs planifiés	node-cron	Rappels réunions, rapports, sauvegardes, fermeture auto réunions

2.2 Structure du projet

Backend (D:/Gestion planning/backend/)

Chemin	Contenu
server.js	Point d'entrée : Express, Socket.IO, Swagger, middlewares globaux, montage de toutes les routes
prisma/schema.prisma	Schéma de base de données complet (modèles, relations, enums)
src/routes/	auth.js, plannings.js, meetings.js, missions.js, rooms.js, users.js, notifications.js, projects.js, dashboard.js, directMessages.js, directionMessages.js, calendar.js, profile.js, 2fa.js, audit.js, push.js, superAdmin.js, adminSettings.js
src/services/	notification.service.js (in-app + email), email.service.js (Nodemailer)
src/middlewares/	auth.middleware.js (JWT), role.middleware.js (RBAC)
src/utils/	logger.js (3 loggers Winston), roles.js
src/config/	swagger.js (Swagger UI config)
src/jobs/	Cron jobs : rappels réunions, rapports hebdo, backup, digest, fermeture réunions

Frontend (D:/Gestion planning/frontend/src/)

Chemin	Contenu
api/client.js	Axios instance avec injection automatique du token JWT
api/notifications.js	Wrappers d'appels API pour les notifications
store/notificationStore.js	Store Zustand : état des notifications, compteur non-lus
context/AuthContext.jsx	État d'authentification global : user, tokens, login/logout
context/ThemeModeContext.jsx	Toggle dark/light mode avec persistance localStorage
realtime/socket.js	Client Socket.IO : initialisation, event listeners
pages/	Toutes les pages React (voir section Modules)
components/	AppLayout, Navbar, NotificationBell, NotificationPanel, UserAvatar, PWAIInstallPrompt, OfflineBanner
utils/roles.js	Utilitaires RBAC côté frontend : isPrivilegedAdmin, hasRole, etc.

2.3 Flux de communication

1. Requête HTTP classique : Browser → Axios (JWT Authorization header) → Express → Prisma → PostgreSQL → réponse JSON
2. Temps réel : Browser ↔ Socket.IO Server (namespace /) avec authentification par JWT dans le handshake
3. Email : Express → Nodemailer → Serveur SMTP externe → boîte email destinataire
4. Push mobile : Express → Firebase Admin SDK → FCM → appareil Android/iOS/Web
5. Fichiers : Upload → Multer (validation type/taille) → stockage /uploads/ → servi via express.static

3. Modèle de Données (Base de Données)

Base de données : PostgreSQL. ORM : Prisma. Toutes les migrations sont versionnées dans `prisma/migrations/`.

3.1 Modèles Utilisateurs & Authentification

User

Champ	Type	Description
id	String (cuid)	Identifiant unique
name	String	Nom d'affichage
email	String (unique)	Adresse email — identifiant de connexion
passwordHash	String	Mot de passe haché bcrypt (salt=12)
role	Enum	RESPONSABLE CONSOLIDATEUR DG ADMIN SUPER_ADMIN
isActive	Boolean	Compte actif (défaut: true)
isDeleted	Boolean	Suppression logique (soft delete, défaut: false)
avatarUrl	String?	URL de l'avatar uploadé
twoFactorEnabled	Boolean	2FA TOTP activée (défaut: false)
twoFactorSecret	String?	Secret TOTP (chiffré en base)
createdAt / updatedAt	DateTime	Horodatages automatiques Prisma

RefreshToken

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	String	FK → User.id
token	String (unique)	Token JWT refresh (haché SHA-256 en base)
expiresAt	DateTime	Date d'expiration (défaut: +7j)
isRevoked	Boolean	Révoqué lors du logout

PasswordHistory

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	String	FK → User.id
passwordHash	String	Hash bcrypt du mot de passe historisé

Champ	Type	Description
createdAt	DateTime	Date de création

Règle métier : les 3 derniers mots de passe sont interdits lors d'un changement (CDC §3.1.2).

DeviceToken (push mobile)

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	String	FK → User.id
token	String (unique)	Token FCM de l'appareil
platform	String	android ios web
createdAt	DateTime	Date d'enregistrement

3.2 Modèles Planning

Planning

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	String	FK → User.id (propriétaire)
weekStart	DateTime	Lundi de la semaine (date ISO)
status	Enum	DRAFT SUBMITTED IN_CONSOLIDATION VALIDATED RETURNED
submittedAt	DateTime?	Date de soumission
consolidatedAt	DateTime?	Date de consolidation
validatedAt	DateTime?	Date de validation DG
returnedAt	DateTime?	Date de retour
returnComment	String?	Commentaire de retour du DG

PlanningEvent

Champ	Type	Description
id	String (cuid)	Identifiant unique
planningId	String	FK → Planning.id
title	String	Titre de l'événement
type	String	MEETING MISSION TRAINING ADMIN OTHER
startTime / endTime	DateTime	Plage horaire de l'événement

Champ	Type	Description
roomId	String?	FK → Room.id (optionnel)
directionId	String?	FK → Direction.id (optionnel)
projectId	String?	FK → Project.id (optionnel)
destination	String?	Lieu (pour missions)
description	String?	Description libre

3.3 Modèles Réunions

Meeting

Champ	Type	Description
id	String (cuid)	Identifiant unique
title	String	Titre de la réunion
agenda	String?	Agenda / description
organizerId	String	FK → User.id
roomId	String?	FK → Room.id
directionId	String?	FK → Direction.id
projectId	String?	FK → Project.id
meetingLink	String?	Lien visioconférence
startTime / endTime	DateTime	Plage horaire
status	Enum	DRAFT SENT CONFIRMED CANCELLED
orderOfDay	String?	Ordre du jour structuré
attachments	String[]	URLs des pièces jointes

Invitation

Champ	Type	Description
id	String (cuid)	Identifiant unique
meetingId	String	FK → Meeting.id
userId	String	FK → User.id (invité)
status	Enum	PENDING ACCEPTED DECLINED
sentAt	DateTime?	Date d'envoi de l'invitation
respondedAt	DateTime?	Date de réponse

3.4 Modèles Missions

Mission

Champ	Type	Description
id	String (cuid)	Identifiant unique
title	String	Titre de la mission
description	String?	Description détaillée
location	String?	Lieu de la mission
startTime / endTime	DateTime	Dates de début et fin
directionId	String?	FK → Direction.id
projectId	String?	FK → Project.id
createdById	String	FK → User.id (créateur)
status	Enum	CONFIRMED CANCELLED

MissionAssignment

Champ	Type	Description
id	String (cuid)	Identifiant unique
missionId	String	FK → Mission.id
userId	String	FK → User.id (membre assigné)

3.5 Modèles Salles

Room

Champ	Type	Description
id	String (cuid)	Identifiant unique
name	String	Nom de la salle
capacity	Int	Capacité (nombre de places)
location	String?	Localisation (bâtiment/étage)
equipment	String[]	Liste des équipements
openFrom / openTo	String?	Horaires d'ouverture (ex: 08:00 / 18:00)
status	Enum	ACTIVE INACTIVE

RoomBooking

Champ	Type	Description
id	String (cuid)	Identifiant unique
roomId	String	FK → Room.id

Champ	Type	Description
meetingId	String	FK → Meeting.id
userId	String	FK → User.id (réservant)
date	DateTime	Date de la réservation
startTime / endTime	DateTime	Plage horaire réservée
status	Enum	CONFIRMED

3.6 Modèles Messagerie

DirectMessage

Champ	Type	Description
id	String (cuid)	Identifiant unique
senderId	String	FK → User.id
receiverId	String	FK → User.id
parentId	String?	FK → DirectMessage.id (thread)
body	String?	Corps du message
fileName / fileUrl	String?	Pièce jointe
isRead	Boolean	Statut de lecture
createdAt	DateTime	Horodatage

DirectionMessage

Champ	Type	Description
id	String (cuid)	Identifiant unique
directionId	String	FK → Direction.id
senderId	String	FK → User.id
parentId	String?	FK → DirectionMessage.id (thread)
body	String?	Corps du message
fileName / fileUrl	String?	Pièce jointe

MeetingMessage

Champ	Type	Description
id	String (cuid)	Identifiant unique
meetingId	String	FK → Meeting.id
senderId	String	FK → User.id
parentId	String?	FK → MeetingMessage.id (thread)

Champ	Type	Description
body	String	Corps du message

3.7 Modèles Organisationnels

Direction

Champ	Type	Description
id	String (cuid)	Identifiant unique
name	String	Nom de la direction
code	String (unique)	Code court de la direction
logoUrl	String?	URL du logo
description	String?	Description
isActive	Boolean	Active / archivée

Project

Champ	Type	Description
id	String (cuid)	Identifiant unique
name	String	Nom du projet
code	String (unique)	Code projet
description	String?	Description
status	Enum	ACTIVE PAUSED COMPLETED
isActive	Boolean	Actif / archivé
createdById	String	FK → User.id

3.8 Modèles Système

Notification

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	String	FK → User.id (destinataire)
type	String	Type de notification (ex: MEETING_INVITE)
title	String	Titre affiché
body	String	Corps du message
link	String?	URL de redirection au clic

Champ	Type	Description
isRead	Boolean	Lu / non lu
createdAt	DateTime	Horodatage

AuditLog

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	String?	FK → User.id (auteur de l'action)
action	String	Type d'action (CREATE, UPDATE, DELETE, LOGIN, etc.)
entity	String	Entité concernée (Meeting, User, Planning, etc.)
entityId	String?	Identifiant de l'entité concernée
ipAddress	String?	Adresse IP de l'auteur
details	JSON?	Détails supplémentaires (champs modifiés, etc.)
createdAt	DateTime	Horodatage

AppSetting

Champ	Type	Description
key	String (PK)	Clé du paramètre (ex: app_name, 2fa_enabled)
value	String	Valeur (JSON serialisé ou texte brut)
updatedAt	DateTime	Dernière mise à jour

Backup

Champ	Type	Description
id	String (cuid)	Identifiant unique
fileName	String	Nom du fichier .sql
relativePath	String	Chemin relatif dans /backups/
sizeBytes	Int	Taille en octets
status	Enum	PENDING SUCCESS FAILED
kind	Enum	MANUAL SCHEDULED
createdById	String?	FK → User.id (déclencheur manuel)
createdAt	DateTime	Horodatage

4. API REST — Endpoints par Module

Base URL : <http://localhost:3001/api> | Documentation interactive : <http://localhost:3001/api/docs> (Swagger UI)

4.1 Authentification — /api/auth

Méthode	Endpoint	Description	Auth requise
POST	/login	Connexion email + mot de passe, retourne access + refresh tokens	Non
POST	/2fa-login	Vérification code TOTP, retourne tokens définitifs	tempToken (5min)
POST	/refresh	Renouveler l'access token via refresh token	refreshToken
POST	/logout	Révoquer le refresh token	JWT
POST	/forgot-password	Demande de réinitialisation par email	Non
POST	/reset-password	Réinitialiser le mot de passe via token email	resetToken
POST	/activate-account	Activer un compte utilisateur créé par l'Admin	activationToken

4.2 Double Authentification — /api/2fa

Méthode	Endpoint	Description	Auth requise
GET	/status	Statut 2FA de l'utilisateur connecté	JWT
POST	/enable	Activer la 2FA (retourne QR code + secret)	JWT
POST	/verify	Vérifier le code TOTP pour finaliser l'activation	JWT
POST	/disable	Désactiver la 2FA	JWT

4.3 Profil — /api/profile

Méthode	Endpoint	Description	Auth requise
GET	/	Obtenir le profil de l'utilisateur connecté	JWT
PUT	/	Modifier nom et email	JWT

Méthode	Endpoint	Description	Auth requise
POST	/avatar	Uploader une photo de profil	JWT
PUT	/password	Changer le mot de passe (validation historique)	JWT

4.4 Plannings — /api/plannings

Méthode	Endpoint	Description	Rôle minimum
GET	/week/:date	Plannings de la semaine (filtrable ?mine=1)	RESPONSABLE
GET	/admin/list	Tous les plannings avec pagination et filtres	ADMIN
POST	/	Créer un planning pour la semaine	RESPONSABLE
POST	/:planningId/events	Ajouter un événement à un planning	RESPONSABLE
PUT	/:eventId	Modifier un événement	RESPONSABLE (propriétaire)
DELETE	/:eventId	Supprimer un événement	RESPONSABLE (propriétaire)
POST	/:planningId/submit	Soumettre le planning	RESPONSABLE
POST	/:planningId/consolidate	Consolider le planning	CONSOLIDATEUR
POST	/:planningId/validate	Valider le planning	DG
POST	/:planningId/return	Retourner avec commentaire	DG

4.5 Réunions — /api/meetings

Méthode	Endpoint	Description	Rôle minimum
GET	/	Liste des réunions de l'utilisateur	RESPONSABLE
GET	/:id	Détail d'une réunion	Participant
POST	/	Créer une réunion (vérif. conflits)	RESPONSABLE
PUT	/:id	Modifier une réunion	Organisateur
DELETE	/:id	Supprimer une réunion	Organisateur
POST	/:id/send	Envoyer les invitations	Organisateur
POST	/:id/accept	Accepter l'invitation	Invité
POST	/:id/decline	Refuser l'invitation	Invité
POST	/:id/files	Uploader une pièce jointe	Participant

Méthode	Endpoint	Description	Rôle minimum
POST	/:id/messages	Envoyer un message dans la réunion	Participant
GET	/:id/messages	Historique des messages	Participant
DELETE	/messages/:msgId	Supprimer un message	SUPER_ADMIN

4.6 Missions — /api/missions

Méthode	Endpoint	Description	Rôle minimum
GET	/	Liste des missions de l'utilisateur	RESPONSABLE
GET	/:id	Détail d'une mission	RESPONSABLE
POST	/	Créer une mission	RESPONSABLE
PUT	/:id	Modifier une mission	Créateur
DELETE	/:id	Supprimer une mission	Créateur
POST	/:id/assign	Affecter un utilisateur	Créateur
DELETE	/:id/assign/:userId	Retirer un utilisateur	Créateur
POST	/:id/files	Uploader un document	Participant

4.7 Salles — /api/rooms

Méthode	Endpoint	Description	Rôle minimum
GET	/	Liste des salles actives	RESPONSABLE
GET	/status	Statut d'occupation en temps réel	RESPONSABLE
GET	/:id/availability	Disponibilité sur une plage horaire	RESPONSABLE
POST	/:id/book	Réserver une salle pour une réunion	RESPONSABLE
POST	/	Créer une salle	ADMIN
PUT	/:id	Modifier une salle	ADMIN
DELETE	/:id	Supprimer une salle	ADMIN

4.8 Projets — /api/projects

Méthode	Endpoint	Description	Rôle minimum
GET	/	Liste des projets	RESPONSABLE
GET	/:id	Détail d'un projet	RESPONSABLE
POST	/	Créer un projet	ADMIN / DG

Méthode	Endpoint	Description	Rôle minimum
PUT	/:id	Modifier un projet	ADMIN / DG
DELETE	/:id	Supprimer un projet	ADMIN / DG
POST	/:id/files	Uploader un document	RESPONSABLE
GET	/:id/files	Liste des fichiers du projet	RESPONSABLE

4.9 Messagerie Directe — /api/direct-messages

Méthode	Endpoint	Description	Rôle minimum
GET	/conversations	Liste des conversations de l'utilisateur	RESPONSABLE
GET	/thread/:userId	Messages avec un utilisateur spécifique	RESPONSABLE
POST	/	Envoyer un message (texte ou fichier)	RESPONSABLE
POST	/:id/reply	Répondre à un message (thread)	RESPONSABLE
PUT	/:id/read	Marquer comme lu	RESPONSABLE
POST	/audit/conversations	Toutes les conversations (modération)	SUPER_ADMIN
DELETE	/audit/messages/:id	Supprimer un message (modération)	SUPER_ADMIN

4.10 Notifications — /api/notifications

Méthode	Endpoint	Description	Rôle minimum
GET	/	Liste paginée des notifications	RESPONSABLE
GET	/unread/count	Nombre de notifications non lues	RESPONSABLE
POST	/:id/read	Marquer une notification comme lue	RESPONSABLE
POST	/mark-all-read	Tout marquer comme lu	RESPONSABLE
GET	/preferences	Préférences de notifications de l'utilisateur	RESPONSABLE
PUT	/preferences	Mettre à jour les préférences	RESPONSABLE
POST	/send	Envoyer une notification manuelle	ADMIN

4.11 Administration — /api/users & /api/admin

Méthode	Endpoint	Description	Rôle minimum
GET	/users	Liste tous les utilisateurs	ADMIN
POST	/users	Créer un utilisateur (email d'activation auto)	ADMIN
PUT	/users/:id	Modifier un utilisateur	ADMIN
DELETE	/users/:id	Suppression logique	ADMIN
PUT	/users/:id/activate	Activer / désactiver	ADMIN
PUT	/users/:id/role	Changer le rôle	ADMIN
GET	/admin/settings	Paramètres de l'application	ADMIN
PUT	/admin/settings	Mettre à jour les paramètres	ADMIN
GET	/audit-logs	Journaux d'audit paginés	ADMIN
GET	/admin/directions	Liste des directions	ADMIN / DG
POST	/admin/directions	Créer une direction	ADMIN
PUT	/admin/directions/:id	Modifier une direction	ADMIN
GET	/dashboard/admin-stats	Statistiques globales du système	ADMIN

4.12 Super Admin — /api/super-admin

Méthode	Endpoint	Description	Rôle minimum
GET	/backups	Liste des sauvegardes	SUPER_ADMIN
POST	/backups	Déclencher une sauvegarde manuelle	SUPER_ADMIN
GET	/backups/:id/download	Télécharger une sauvegarde	SUPER_ADMIN
POST	/backups/:id/restore	Restaurer depuis une sauvegarde	SUPER_ADMIN
DELETE	/backups/:id	Supprimer une sauvegarde	SUPER_ADMIN

5. Fonctionnement Temps Réel — Socket.IO

5.1 Architecture Socket.IO

Le serveur Socket.IO partage la même instance HTTP qu'Express. L'authentification se fait via le token JWT transmis dans le handshake (auth.token). Chaque connexion est associée à un userId.

5.2 Événements émis par le serveur

Événement	Destinataire	Déclencheur
socket:ready	Utilisateur connecté	Connexion Socket.IO établie
presence:online	Tous les utilisateurs	Un utilisateur se connecte
presence:offline	Tous les utilisateurs	Un utilisateur se déconnecte
presence:sync	Utilisateur connecté	Liste des utilisateurs en ligne à la connexion
notification:new	Utilisateur cible	Création d'une nouvelle notification
meeting:update	Participants de la réunion	Modification, annulation d'une réunion
message:direct	Destinataire du message	Nouveau message direct reçu
message:direction	Membres de la direction	Nouveau message de direction
message:meeting	Participants de la réunion	Nouveau message dans la réunion

5.3 Événements émis par le client

Événement	Description
meeting:join	Rejoindre la room Socket.IO d'une réunion pour recevoir les mises à jour
meeting:leave	Quitter la room Socket.IO d'une réunion

6. Authentification & Sécurité

6.1 Flux d'authentification JWT

6. L'utilisateur soumet email + mot de passe → POST /api/auth/login
7. Le serveur vérifie le hash bcrypt (salt=12), puis vérifie si la 2FA est requise
8. Sans 2FA : retourne accessToken (15min) + refreshToken (7j) + objet user
9. Avec 2FA : retourne twoFactorRequired=true + tempToken (JWT 5min à usage unique)
10. L'utilisateur soumet le code TOTP → POST /api/auth/2fa-login avec tempToken
11. Le serveur vérifie le TOTP (speakeasy), blackliste le tempToken → retourne les tokens définitifs

6.2 Gestion des tokens

Token	Durée	Stockage
Access Token	15 minutes	localStorage (frontend), Authorization: Bearer header
Refresh Token	7 jours	localStorage (frontend), DB (isRevoked flag)
Reset Password Token	Durée limitée	Hash SHA-256 en DB
Activation Token	Durée limitée	Hash SHA-256 en DB
Temp Token 2FA	5 minutes	JWT signé + blacklist en mémoire

6.3 Middleware d'authentification

auth.middleware.js extrait le token JWT du header Authorization: Bearer <token>, vérifie la signature avec JWT_SECRET, récupère l'utilisateur en base, vérifie que isActive=true et isDeleted=false.

6.4 Contrôle d'accès basé sur les rôles (RBAC)

role.middleware.js vérifie que le rôle de l'utilisateur est dans la liste des rôles autorisés passée en paramètre. L'ordre de priorité des rôles :

Rôle	Niveau	Accès additionnels
RESPONSABLE	1	Modules métier de base (planning, réunions, missions, salles)
CONSOLIDATEUR	2	Consolidation des plannings d'équipe
DG	3	Validation/retour des plannings, gestion des Directions
ADMIN	4	Panneau d'administration complet
SUPER_ADMIN	5	Sauvegardes, modération messages, promotion SUPER_ADMIN

6.5 Protection brute-force

Endpoint	Limite	Durée de blocage	Identifiant de blocage
POST /auth/login	5 tentatives	15 minutes	email + IP
POST /auth/2fa-login	5 tentatives	15 minutes	tempToken
API générale	200 requêtes/min	1 minute	IP

6.6 Headers de sécurité (Helmet.js)

- HSTS (Strict-Transport-Security) — force HTTPS en production
- X-Frame-Options: DENY — protection contre le clickjacking
- X-Content-Type-Options: nosniff — protection MIME type sniffing
- Content-Security-Policy — active partout sauf sous /api/docs (Swagger)
- Referrer-Policy, Permissions-Policy

7. Système de Notifications

7.1 Canaux de notification

Canal	Technologie	Déclenchement
In-app	Socket.IO + DB Notification	Temps réel via socket + persisté pour historique
Email	Nodemailer SMTP	Asynchrone, respecte les préférences utilisateur
Push mobile	Firebase FCM	Via DeviceToken enregistré, Android/iOS/Web

7.2 Préférences utilisateur

- L'utilisateur peut activer/désactiver chaque canal (in-app, email) individuellement
- Heures silencieuses : aucune notification email entre 22h00 et 07h00 (configurable)
- Mode digest : les notifications sont regroupées en un résumé quotidien

7.3 Jobs automatiques (node-cron)

Job	Fréquence	Action
Rappel réunions	Tous les jours à 08h00	Notifie les invités des réunions prévues le lendemain
Rapport hebdomadaire	Tous les lundis à 08h00	Envoie un rapport de stats à WEEKLY_REPORT_EMAIL
Digest notifications	Toutes les heures	Agrège les notifications pour les utilisateurs en mode digest
Fermeture auto réunions	Toutes les 5 minutes	Ferme les réunions dépassant leur heure de fin
Sauvegarde BDD	Tous les dimanches à 00h00	pg_dump PostgreSQL vers /backups/

8. Gestion des Fichiers (Upload)

Les fichiers sont gérés par Multer côté serveur et servis via express.static. Chaque catégorie a son propre répertoire et ses propres contraintes.

Répertoire	Usage	Taille max	Types acceptés
/uploads/avatars	Photos de profil utilisateur	5 Mo	Images (jpg, png, gif, webp)
/uploads/meetings	Pièces jointes de réunions	Variable	Tous types
/uploads/missions	Documents de missions	15 Mo	Tous types
/uploads/project-files	Documents de projets	20 Mo	Tous types
/uploads/direct-messages	Fichiers messages directs	15 Mo	Tous types
/uploads/branding	Logo de l'application	5 Mo	Images
/backups	Sauvegardes pg_dump	Illimité	Fichiers .sql

9. Configuration — Variables d'Environnement

9.1 Variables Backend (.env)

Variable	Requis	Description
DATABASE_URL	Oui	URL de connexion PostgreSQL (format Prisma)
JWT_SECRET	Oui	Clé secrète pour signer les tokens JWT
JWT_EXPIRY	Non	Durée de l'access token (défaut: 15m)
JWT_REFRESH_EXPIRY	Non	Durée du refresh token (défaut: 7d)
FRONTEND_URL	Oui	URL du frontend React (CORS + liens emails)
PORT	Non	Port du serveur (défaut: 3001)
NODE_ENV	Non	production development
SMTP_HOST	Oui	Serveur SMTP pour l'envoi d'emails
SMTP_PORT	Oui	Port SMTP (ex: 587 pour TLS)
SMTP_USER	Oui	Identifiant SMTP
SMTP_PASS	Oui	Mot de passe SMTP
SMTP_SECURE	Non	true pour SSL/TLS direct
SMTP_FROM	Non	Adresse d'expéditeur des emails
FIREBASE_SERVICE_ACCOUNT_JSON	Non	JSON du compte de service Firebase (pour FCM)
MAX_LOGIN_ATTEMPTS	Non	Tentatives de login avant blocage (défaut: 5)
LOGIN_BLOCK_DURATION	Non	Durée de blocage en minutes (défaut: 15)
MAX_2FA_ATTEMPTS	Non	Tentatives 2FA avant blocage (défaut: 5)
WEEKLY_REPORT_EMAIL	Non	Email destinataire du rapport hebdomadaire
BACKUP_CRON_TIMEZONE	Non	Timezone pour le cron de sauvegarde
DISABLE_BACKUP_CRON	Non	true pour désactiver la sauvegarde automatique
SWAGGER_USER / SWAGGER_PASS	Non	Credentials Basic Auth pour Swagger en production

9.2 Variables Frontend (.env)

Variable	Requis	Description
VITE_API_URL	Oui	URL du backend Express (ex: http://localhost:3001)
VITE_SOCKET_URL	Non	URL Socket.IO (défaut: VITE_API_URL)
VITE_FIREBASE_*	Non	Configuration Firebase pour les notifications push web

10. Logging & Observabilité

10.1 Loggers Winston

Logger	Fichier de sortie	Usage
logger	logs/application.log	Logs applicatifs généraux (info, warn, error)
auditLogger	logs/audit.log	Journalisation des actions métier sensibles
errorLogger	logs/errors.log	Erreurs non gérées et exceptions critiques

10.2 Middleware de logging des requêtes

Un middleware dans server.js enregistre chaque requête HTTP avec : méthode, chemin, code de statut, durée d'exécution (ms), userId si authentifié.

10.3 Audit Trail (AuditLog)

Toutes les actions importantes sont enregistrées en base dans le modèle AuditLog. Les routes font appel à `prisma.auditLog.create()` à chaque opération significative.

11. Workflows Métier Détaillés

11.1 Workflow de Validation des Plannings

Ce workflow est le processus central de l'application. Il implique 3 niveaux d'acteurs et peut comporter des aller-retours.

Étape	Acteur	Action	Résultat
1	RESPONSABLE	Crée des événements et soumet le planning	Statut → SUBMITTED
2	CONSOLIDATEUR	Révisé et consolide les plannings soumis	Statut → IN_CONSOLIDATION
3	DG	Valide le planning consolidé	Statut → VALIDATED
3 bis	DG	Retourne avec commentaire si modifications nécessaires	Statut → RETURNED
4	RESPONSABLE	Modifie et resoumet	Statut → SUBMITTED (retour à l'étape 2)

11.2 Workflow d'Organisation d'une Réunion

12. L'organisateur crée la réunion (statut DRAFT)
13. Il ajoute les participants (vérification des conflits horaires)
14. Il sélectionne une salle (vérification de disponibilité)
15. Il envoie les invitations : emails automatiques + notifications in-app → statut SENT
16. Les invités acceptent ou refusent (l'organisateur est notifié à chaque réponse)
17. La réunion se tient. Le cron job auto-ferme la réunion à l'heure de fin → statut CONFIRMED
18. L'organisateur peut rouvrir la réunion si nécessaire

11.3 Workflow d'Activation d'un Compte

19. L'Admin crée un compte utilisateur (POST /api/users)
20. Le serveur génère un token d'activation (hash SHA-256) et envoie un email
21. L'utilisateur clique sur le lien dans l'email → page /activate-account
22. Il définit son mot de passe initial (validé contre l'historique)
23. Le compte est activé (isActive=true, token consommé)
24. L'utilisateur peut désormais se connecter

12. Déploiement & Infrastructure

12.1 Prérequis

- Node.js 22.x LTS
- PostgreSQL 14+ (ou 15/16)
- npm 10+ ou yarn
- Compte Firebase (pour les notifications push mobiles)
- Serveur SMTP (pour les notifications email)

12.2 Installation et démarrage

Backend :

```
cd backend
npm install
npx prisma migrate deploy
npm run start
```

Frontend :

```
cd frontend
npm install
npm run build      # production
npm run dev        # développement (port 5173)
```

12.3 Ports et URLs

Service	Port par défaut	URL
Backend API	3001	http://localhost:3001/api
Swagger UI	3001	/api/docs
Frontend	5173	http://localhost:5173
Socket.IO	3001 (shared)	ws://localhost:3001

12.4 Migrations de base de données

Les migrations Prisma sont versionnées dans backend/prisma/migrations/. En production, utiliser `npx prisma migrate deploy` (sans réinitialisation des données). En développement, `npx prisma migrate dev` crée automatiquement les nouvelles migrations à partir des changements de schéma.

13. Glossaire

Terme	Définition
JWT (JSON Web Token)	Standard d'authentification stateless — token signé contenant les claims de l'utilisateur
TOTP	Time-based One-Time Password — code à 6 chiffres renouvelé toutes les 30 secondes (RFC 6238)
Socket.IO	Bibliothèque de communication bidirectionnelle en temps réel basée sur WebSocket
Prisma ORM	Object-Relational Mapper pour Node.js — génère un client typé à partir du schéma
FCM	Firebase Cloud Messaging — service de push notifications Google (Android/iOS/Web)
PWA	Progressive Web App — application web installable sur mobile avec capacités natives
Soft delete	Suppression logique : le record est marqué isDeleted=true mais pas effacé physiquement
Bcrypt	Algorithme de hachage de mots de passe avec facteur de coût (salt rounds)
Rate limiting	Limitation du nombre de requêtes par unité de temps pour prévenir les abus
Webhook / Cron	Tâche planifiée exécutée à intervalles réguliers côté serveur (node-cron)
RBAC	Role-Based Access Control — contrôle d'accès basé sur les rôles utilisateurs
CSP	Content Security Policy — directive HTTP limitant les ressources autorisées à charger
pg_dump	Outil PostgreSQL de sauvegarde d'une base de données en fichier SQL
SMTP	Simple Mail Transfer Protocol — protocole standard d'envoi d'emails
CDC	Cahier des Charges — document de référence des spécifications du projet

Gestion Planning — Document Technique & Fonctionnel

Version 1.0 | Avril 2026 | Al hussein khouma